Design & Implementation of a DBMS for Campus Facility Maintenance Request Management

Review 3 · Rimika Jain: 25WU0101111: AIMLRhinos'29: DBMS PBL

REVIEW 3 DELIVERABLES (WEEK 16 - 5 MARKS)

- Database-Connected Application Development (Python & Relational RDBMS)
- End-to-End CRUD Workflows
- Business Rules Validation & Error Handling
- Analytical Reports & Viva-Voce Defense Readiness

System Architecture: 3-Tier Enterprise Design

Tier 1: Presentation Layer

Interactive Graphical & Console User Interfaces

- **Requester Portal:** Allows students/staff to submit issues, attach room info, and view live status.
- **Supervisor Dispatch:** Office dashboard for assigning technicians, setting SLAs, and tracking backlog.
- **Technician Mobile/Terminal:** Work log recording, parts consumption entry, and completion sign-off.
- **Executive Analytics UI:** Graphical reports, KPI dials, cost charts, and rating summaries.

Tier 2: Business Logic & Application Layer

Python Application Engine & Controller

- **Validation Controller:** Guards input integrity, date logic, and sanitizes strings against SQL injection.
- **State Machine Engine:** Strict status transitions: Open → Assigned → In Progress → Completed.
- **Transaction Guard:** Atomic commit/rollback for inventory deduction and cost ledger posting.
- **Database Connector:** Pooled connections via sqlite3 / mysql-connector-python with parameterization.

Tier 3: Database & Persistence Layer

Normalized Relational RDBMS (MySQL / SQLite)

- **15 Relational Tables:** Normalized to 3NF, eliminating redundancy and update anomalies.
- **Active Constraints:** PK, FK (RESTRICT/CASCADE), UNIQUE codes, and CHECK validation rules.
- **Optimized Indexes:** B-Tree indexes on RequestCode, UserID, and RoomID for sub-5ms queries.
- **Stored Views:** Virtual views for executive dashboarding and operational health metrics.

REVIEW 3 · APPLICATION & FINAL DEMONSTRATION

Application Modules & Relational Database Mapping

Application Module	Underlying Tables	Primary & Foreign Keys	SQL Operations & Transactions

M1: Requester Portal	Users, Requests, Rooms, Buildings, Assets	PK: RequestID, UserID
		FK: UserID, RoomID, AssetID
			`INSERT INTO Requests;`
			`SELECT historical requests by UserID;`
			`DELETE unassigned requests (Cancel).`

M2: Facility Office Dispatch	Requests, Technicians, Assignments, Priority	PK: AssignmentID
		FK: RequestID, TechID, PriorityID
			`UPDATE Requests SET PriorityID=...;`
			`INSERT INTO Assignments;`
			`UPDATE Technicians SET AvailabilityStatus='Busy'.`

M3: Service & Work Log	Assignments, Work_Logs, Request_Materials, Materials	PK: LogID, (LogID, MaterialID)
		FK: AssignmentID, MaterialID
			`INSERT INTO Work_Logs;`
			`INSERT INTO Request_Materials;`
			`UPDATE Materials SET QuantityInStock = Stock - Used.`

M4: Job Closure & Costing	Requests, Costs, Status_History	PK: CostID, StatusLogID
		FK: RequestID
			`UPDATE Requests SET CurrentStatus='Completed';`
			`INSERT INTO Costs (Material, Labor);`
			`INSERT INTO Status_History.`

M5: Feedback & Analytics	Feedback, v_FacilityHealthDashboard	PK: FeedbackID FK: RequestID	`INSERT INTO Feedback (Rating, Comments);` `SELECT * FROM views for KPI reporting.`

REVIEW 3 · APPLICATION & FINAL DEMONSTRATION

Full CRUD Operations Demonstration

CREATE (C): Complaint Logging

Student or faculty logs a new facility breakdown.

- **Input Form:** Select Block, Room (e.g. Lab 204), Asset (AC Unit), describe fault.
- **SQL Action:** `INSERT INTO Requests (RequestCode, UserID, RoomID, AssetID, Description, PriorityID, CurrentStatus) VALUES (?, ?, ?, ?, ?, ?, 'Open')`
- **Validation:** Auto-generates sequential tracking code 'REQ-2026-0042' and verifies user session.

READ (R): Multi-Criteria Lookup

Real-time query retrieval for requesters and managers.

- **User Action:** View active tickets or query by ticket number, technician, or building.
- **SQL Action:** `SELECT r.RequestCode, b.BuildingName, rm.RoomNumber, r.CurrentStatus, p.LevelName FROM Requests r JOIN Rooms rm ... WHERE r.CurrentStatus = ?`
- **Capability:** Instant dynamic filtering with B-Tree indexes yielding sub-5ms responses.

UPDATE (U): Dispatch & Work Entry

Supervisor assigns staff; technician records labour and materials.

- **Work Order:** Technician logs 2.5 labour hours and 1 replacement compressor capacitor.
- **SQL Action:** `UPDATE Requests SET CurrentStatus='In Progress' WHERE RequestID=?; INSERT INTO Work_Logs ...; UPDATE Materials SET QuantityInStock = QuantityInStock - 1;`
- **Atomicity:** Executed inside a single database transaction block with rollback protection.

DELETE / CANCEL (D): Safe Closure

Requesters cancel mistakenly filed issues or soft-delete records.

- **Policy Rule:** Requests cannot be physically deleted if work logs or costs exist (RESTRICT FK).
- **SQL Action:** `UPDATE Requests SET CurrentStatus='Cancelled' WHERE RequestID=? AND CurrentStatus='Open'`
- **Audit Trail:** Preserves audit trail and logs cancellation reason in Status_History table.

REVIEW 3 · APPLICATION & FINAL DEMONSTRATION

Dynamic Search & Multi-Parameter Filter Workflows

1. Multi-Parameter Search Engine

Enables facility administrators to filter unresolved complaints by building, priority, date range, or technician with dynamic SQL parameter building.

```
# Dynamic Parameterized Search in Python
query = '''SELECT r.RequestCode, b.BuildingName,
                  r.Description, r.CurrentStatus
           FROM Requests r
           JOIN Rooms rm ON r.RoomID = rm.RoomID
           JOIN Buildings b ON rm.BuildingID = b.BuildingID
           WHERE 1=1'''
params = []
if building_id:
    query += ' AND b.BuildingID = ?'; params.append(building_id)
if status_val:
    query += ' AND r.CurrentStatus = ?'; params.append(status_val)
cursor.execute(query, tuple(params))
```

Search Capabilities & Defensive Benefits:

- Zero SQL Injection Risk: Input sanitization via tuple bindings.
- Wildcard Text Search: Full-text matching on issue description.
- Composite Drill-Down: Combines location, status, and urgency.

2. Technician Dispatch Queue & Load Balancer

Dispatches tickets to available technicians matching required trade specialization while preventing staff workload saturation.

```
-- Smart Technician Recommendation Query
SELECT t.TechID, u.FullName, t.Specialization,
       COUNT(a.AssignmentID) AS ActiveJobCount
FROM Technicians t
JOIN Users u ON t.UserID = u.UserID
LEFT JOIN Assignments a ON t.TechID = a.TechID
     AND a.Status = 'In Progress'
WHERE t.AvailabilityStatus = 'Available'
  AND t.Specialization = ?
GROUP BY t.TechID, u.FullName, t.Specialization
HAVING COUNT(a.AssignmentID) < 3 -- Capacity ceiling
ORDER BY ActiveJobCount ASC;
```

Capacity-Aware Dispatch Result:

- Prevents allocating jobs to overloaded staff (Max 3 active jobs).
- Matches 'HVAC' faults to certified HVAC technicians.
- Updates technician status to 'Busy' when ceiling is reached.

REVIEW 3 · APPLICATION & FINAL DEMONSTRATION

Business Rules, Defensive Validation & ACID Transactions

1. Status Lifecycle State Machine

Enforces legal progression. System rejects jumping directly from 'Open' to 'Completed'. Tickets MUST follow: Open → Assigned → In Progress → Completed. Cancellations permitted only prior to technician dispatch.

2. SQL Injection Immunity

Strictly enforced parameterized queries across all database access layers. Disallows string concatenation in SQL statements; all user inputs bound as typed parameters (? placeholders).

3. Inventory Stock & Cost Integrity

Deducting inventory and adding maintenance expense records is wrapped in a DB transaction: BEGIN TRANSACTION ... UPDATE Stock ... INSERT Cost ... COMMIT. If stock is insufficient, triggers ROLLBACK.

4. Application Exception Handling

Robust Python try-except-finally blocks catch sqlite3.IntegrityError and display meaningful user alerts rather than crashing. Graceful connection closing in finally blocks guarantees zero memory leaks.

Executive Reports & Operational Analytics

Report 1: Request Ageing & SLA Backlog

Identifies tickets open > 48h grouped by urgency.

- Emergency: 2 tickets open (Avg Age: 14.5 hrs)
- High Priority: 4 tickets open (Avg Age: 38.2 hrs)
- Medium / Low: 7 tickets open (Avg Age: 61.0 hrs)
- Action: Auto-alerts facility manager on SLA breaches.

Report 2: Building Maintenance Expenditure

Aggregates spare parts & labor per campus block.

- Academic Block A: ₹18,450 (14 jobs executed)
- Science Complex: ₹12,800 (9 jobs executed)
- Hostel Cluster: ₹24,300 (22 jobs executed)
- Action: Direct input for quarterly university budget allocation.

Report 3: Technician Workload & Turnaround

Analyzes productivity and resolution speed.

- Ramesh Sharma (Elec): 18 completed, 2.1 hr avg fix
- Suresh Verma (HVAC): 12 completed, 3.4 hr avg fix
- Priya Patel (Plumb): 15 completed, 1.9 hr avg fix
- Action: Evaluates technician performance and staffing needs.

Report 4: Inventory & Requester CSAT

Spare parts reorder warnings and customer satisfaction.

- **Stock Alert: 2.5uF AC Capacitors below reorder level (Qty: 2)**
- **Stock Alert: LED Tube lights below reorder level (Qty: 4)**
- Average User Satisfaction: 4.6 / 5.0 (Across 45 reviews)
- Action: Auto-generates purchase requisitions for storekeeper.

REVIEW 3 · APPLICATION & FINAL DEMONSTRATION

Application User Interface & Screen Workflows

Screen 1: Request Submission Portal

Clean, responsive complaint submission form.

- ✓ User Details: Pre-fills Roll No. / Staff ID & Name.
- ✓ Location Selectors: Cascading dropdowns (Building → Room).
- ✓ Asset Tagging: Auto-loads assets present in selected room.
- ✓ Visual Status: Displays generated Tracking ID immediately.

Screen 2: Dispatcher Console

Facility office central control center.

- ✓ Unassigned Queue: Real-time table of incoming tickets.
- ✓ Technician Roster: Shows active load & availability badge.
- ✓ 1-Click Dispatch: Assigns technician and sends SMS/Alert.
- ✓ SLA Indicator: Color-coded urgency badges (Red/Amber/Green).

Screen 3: Technician Service Logging

Work log and parts replacement form.

- ✓ Work Notes: Input field for repair summary and diagnostic.
- ✓ Labour Duration: Hours input with range check (0.5 - 24h).
- ✓ Spare Parts Picker: Deducts parts from stock upon logging.
- ✓ Closure Sign-off: Marks ticket 'Completed' with timestamp.

Screen 4: Executive KPI Dashboard

Analytical decision support view.

- ✓ Live Metrics: Total tickets, pending backlog, active repairs.
- ✓ Cost Breakdown: Interactive summary of monthly campus expense.
- ✓ Feedback Stars: Requester ratings and satisfaction trend.
- ✓ Export Button: Generates formatted CSV / PDF report.

Systematic Testing Suite & Validation Results

Test ID	Test Scenario	Input / Action Taken	Expected & Observed System Behavior	Result Status
TC-01	Valid Complaint Creation	Valid user logs room 302 fan noise with High priority.	Record saved; unique REQ-2026-0031 generated; status 'Open'.	**PASSED**
TC-02	Negative Cost Prevention	Technician enters cost = -500 for replacement switch.	Database CHECK constraint chk_cost triggers; UI alerts user.	**PASSED**
TC-03	Invalid Rating Guard	User submits feedback rating = 7 for closed request.	Application input check + CHECK constraint reject value.	**PASSED**
TC-04	Illegal Status Transition	Attempt to mark 'Open' request directly to 'Completed'.	State machine validator blocks jump; requires prior assignment.	**PASSED**
TC-05	Inventory Deduction Rollback	Requesting 10 units when stock = 3 in database.	Stock check fails; transaction rolled back; stock unchanged.	**PASSED**
TC-06	SQL Injection Resistance	Input: ' OR '1'='1' -- in request search box.	Handled as literal parameter string; zero SQL injection breach.	**PASSED**

REVIEW 3 · APPLICATION & FINAL DEMONSTRATION

Viva-Voce Key Defense Questions & Answers

Q1: Why was 3NF chosen over BCNF for this campus database?

3NF guarantees lossless join decomposition and complete preservation of all functional dependencies. In our campus system, every functional dependency is preserved without requiring expensive multi-table join assertions during transaction processing.

Q2: How does the application maintain ACID properties during work logs?

When a technician logs work and uses spare parts, two operations occur: inserting into Request_Materials and updating QuantityInStock in Materials. Wrapping them in a single transaction (conn.commit() / conn.rollback()) ensures Atomicity and Consistency—preventing orphan cost logs or negative stock.

Q3: How are SQL injection vulnerabilities completely prevented?

By eliminating dynamic string formatting in SQL queries (such as f-strings or %s concatenation) and exclusively utilizing parameterized queries (? placeholders). The database engine compiles the SQL execution plan first and treats all user parameters strictly as data values.

Q4: What foreign key referential actions were chosen and why?

ON DELETE RESTRICT is applied on Users and Rooms to prevent accidental deletion of campus entities with active maintenance records. ON DELETE CASCADE is applied on dependent child tables like Costs and Status_History so that if a test request is deleted, child records clean up automatically.

Project Summary, Learnings & Future Enhancements

Comprehensive PBL Deliverables Achieved

- **Review 1 (Week 7):** Problem identification, 15 conceptual entities, comprehensive ER diagram, and initial schema.
- **Review 2 (Week 11):** 3NF normalization, formal FDs, data dictionary, DDL constraints, multi-table joins, views, and prototype.
- **Review 3 (Week 16):** Full-featured Python front-end, database connectivity, complete CRUD, validation state machine, executive reporting, and viva readiness.
- **Core DBMS Learning:** Mastered real-world database design, data integrity enforcement, SQL tuning, transaction safety, and front-end integration.

Limitations & Future Roadmap

➔ **Mobile App with Push Notifications:** Extend system to Android/iOS so technicians receive immediate push notifications and turn-by-turn campus navigation.

➔ **IoT Fault Telemetry Integration:** Connect smart campus sensors (e.g. water tank levels, HVAC thermostats, power meters) to auto-raise complaints without human intervention.

➔ **Predictive Maintenance AI:** Apply machine learning on historical breakdown frequencies to schedule preventive servicing prior to equipment failure.

➔ **Automated Vendor Integration:** Auto-generate purchase orders and quotation requests to external vendors when spare parts hit minimum reorder points.

THANK YOU!

Project Demonstration & Viva-Voce

Rimika Jain | 25WU0101111 | AIMLRhinos'29 | Woxsen University